DevOps L1 Interview

Complete Q&A Reference Guide

Kubernetes · Docker · Terraform · Monitoring · RBAC · Networking

44 Questions | 8 Sections | Structured Answers with Examples

SECTION 1 — Kubernetes: Ingress & Networking

Q1. What is Ingress in Kubernetes?

- Ingress is a Kubernetes API object that manages external HTTP/HTTPS access to services inside the cluster.
- It acts as a Layer-7 (application layer) traffic router — it understands URLs, hostnames, and paths.
- Unlike Services (NodePort/LoadBalancer), Ingress can route traffic to multiple services from a single entry point.
- Ingress can also terminate SSL/TLS, apply rate limiting, and enforce authentication rules when the controller supports it.

■ *Ingress itself is just a spec; it does nothing without an Ingress Controller.*

Q2. How is Ingress different from a Service (NodePort / LoadBalancer)?

- NodePort: exposes a port on every node; blunt, no routing logic, best for dev/testing only.
- LoadBalancer: provisions a cloud LB per service — expensive at scale (one LB = one service).
- Ingress: single entry point (one LB) → routes to many services via rules; far more cost-efficient.
- Ingress operates at L7 (HTTP); Services operate at L4 (TCP/UDP) — Ingress can route by path/host, Services cannot.

■ *Rule of thumb: use Ingress for HTTP workloads; LoadBalancer for raw TCP/UDP or non-HTTP services.*

Q3. How does Ingress handle HTTP vs HTTPS routing?

- HTTP routing: Ingress matches incoming requests by host/path and forwards to backend services — no encryption.
- HTTPS routing: TLS is terminated at the Ingress controller; the backend typically receives plain HTTP (TLS offloading).
- The tls: block in the Ingress spec references a Kubernetes Secret holding the certificate and private key.
- End-to-end TLS (passthrough) is also supported by some controllers (e.g., NGINX with ssl-passthrough annotation).

```
spec:
tls:
- hosts: [api.example.com]
secretName: api-tls-secret
```

Q4. What is an SSL/TLS certificate, and how is it used in Ingress?

- An SSL/TLS certificate binds a domain name to a public key; it enables encrypted HTTPS communication.
- It is issued by a Certificate Authority (CA) — e.g., Let's Encrypt, DigiCert, AWS ACM.
- In Kubernetes: store cert + private key in a Secret of type kubernetes.io/tls.
- The Ingress spec's tls.secretName field tells the controller which secret to use for that host.
- cert-manager is the standard tool to automate issuance and renewal of Let's Encrypt certs inside clusters.

■ *Never commit TLS secrets to Git — use Sealed Secrets or External Secrets Operator.*

Q5. What is an Ingress Controller? Can Kubernetes Ingress work without it?

- An Ingress Controller is the actual implementation that reads Ingress objects and programs the proxy/load balancer.
- Common controllers: NGINX Ingress, Traefik, HAProxy, AWS ALB Ingress Controller, Istio Gateway.
- No — Ingress without a controller is a no-op. The cluster will accept the YAML but traffic will not be routed.
- Each cluster can run multiple controllers; annotations select which controller handles a given Ingress.

■ *EKS tip: use AWS Load Balancer Controller (ALB Ingress) for native AWS integration with WAF and ACM certs.*

Q6. How do you configure host-based routing in Ingress?

- Host-based routing directs traffic based on the HTTP Host header — i.e., different domains → different services.

```
spec:
rules:
- host: app1.example.com
http:
paths:
- path: /
pathType: Prefix
backend:
service:
name: app1-svc
port:
number: 80
```

- Multiple host blocks can exist in one Ingress object — one LB handles all of them.

■ *DNS must resolve each hostname to the Ingress controller's external IP/CNAME.*

Q7. What is path-based routing in Ingress?

- Path-based routing sends traffic to different services based on the URL path under the same hostname.
- Example: /api → backend-service, /web → frontend-service, both under api.example.com.
- pathType options: Prefix (matches prefix), Exact (exact match), ImplementationSpecific.

```
- path: /api
pathType: Prefix
backend:
service:
name: api-svc
port:
number: 8080
```

■ *Order matters in some controllers — more specific paths should be listed first.*

Q8. What are Network Policies in Kubernetes?

- NetworkPolicy is a Kubernetes object that defines rules for pod-to-pod and pod-to-external traffic at L3/L4.
- By default, all pods can talk to all pods — NetworkPolicies add a deny-by-default model selectively.
- Policies are enforced by the CNI plugin (Calico, Cilium, Weave) — not by the Kubernetes control plane itself.
- Rules select pods via podSelector, namespaceSelector, or ipBlock (CIDR).

■ *Flannel does NOT support NetworkPolicy. Use Calico or Cilium in production.*

Q9. How do Network Policies control pod-to-pod communication?

- ingress rules: control which pods/namespaces can send traffic TO the selected pod.
- egress rules: control which destinations the selected pod can send traffic TO.
- If any NetworkPolicy selects a pod, all traffic not explicitly allowed is denied.
- Selector example: podSelector: matchLabels: app: payments — applies the policy only to payment pods.

```
ingress:
- from:
- podSelector:
matchLabels:
app: frontend
ports:
- port: 8080
```

Q10. Difference between Network Policies and Firewall rules?

- Firewall rules: operate at the host/network level (iptables, AWS Security Groups) — VM/node scoped.
- NetworkPolicy: operates at pod/label level inside Kubernetes — tracks pod identity, not just IP.
- NetworkPolicies are dynamic — as pods scale, labels ensure policies auto-apply without IP management.
- Firewalls protect node-to-node or cluster-to-internet; NetworkPolicies protect service-to-service inside the cluster.

■ *Both layers are needed: SGs protect nodes, NetworkPolicies protect pods. Not either/or.*

Q11. What is the role of gateways in Kubernetes networking?

- A Gateway (in Istio or the Gateway API) is a more powerful alternative to Ingress for managing ingress/egress traffic.
- It separates infrastructure concerns (Gateway = which LB/port) from routing concerns (HTTPRoute = which service).
- Supports advanced protocols: TCP, TLS passthrough, gRPC — beyond what standard Ingress handles.
- In Istio: IngressGateway controls north-south traffic; VirtualService/DestinationRule handle internal routing.

■ *Kubernetes Gateway API (gateway.networking.k8s.io) is the successor to Ingress — more expressive and team-friendly.*

Q12. How do you restrict a pod to access only specific external APIs?

- Use an egress NetworkPolicy to whitelist specific IPs or CIDR ranges.
- Step 1: Apply a default-deny-egress policy to the namespace.
- Step 2: Add an allow rule for the specific external IP/CIDR + port.

```
egress:
- to:
- ipBlock:
cidr: 203.0.113.5/32
ports:
```

```
- port: 443
```

• For domain-based filtering (not just IPs), use a service mesh egress gateway (Istio) or a DNS-aware proxy (Squid).

■ *ipBlock CIDR in NetworkPolicy doesn't resolve DNS — you must know the external IP. Use egress gateway for FQDN-based filtering.*

SECTION 2 — Monitoring: Prometheus, Grafana & Logging

Q13. What metrics can be monitored in Kubernetes?

• Resource metrics: CPU, memory, disk I/O, network I/O per pod/node.

• Control plane metrics: API server request rate, etcd latency, scheduler queue depth.

• Application metrics: request rate, error rate, latency (p50/p95/p99), active connections.

• Cluster-level: node readiness, pod eviction rate, PV usage, HPA scaling events.

• Key sources: metrics-server (basic), Prometheus + cAdvisor (full), Datadog/Dynatrace (APM).

Q14. How do you monitor CPU and memory usage of pods?

• Quick check: kubectl top pods -n — requires metrics-server installed.

• For historical data: use Prometheus + kube-state-metrics + node-exporter.

• Useful PromQL queries:

```
# CPU usage per pod
rate(container_cpu_usage_seconds_total{namespace='prod'}[5m])
```

```
# Memory usage
container_memory_working_set_bytes{container!=''}
```

• Set resource requests/limits in pod spec — metrics-server uses these to compare actual vs requested.

■ *kubectl top shows current snapshot only. For trends, always use Prometheus with a retention period configured.*

Q15. What is Prometheus, and how is it used in Kubernetes?

• Prometheus is an open-source monitoring system with a time-series database and a pull-based scrape model.

• It scrapes metrics from /metrics endpoints exposed by pods, nodes, and kube components.

• In Kubernetes: deploy via kube-prometheus-stack Helm chart — bundles Prometheus, Alertmanager, and Grafana.

• ServiceMonitor CRD tells Prometheus which services to scrape (label-based selection).

• PromQL is used to query, aggregate, and alert on time-series data.

■ *In ApnaKart, Prometheus scrapes Spring Boot actuator /metrics endpoints exposed via Micrometer.*

Q16. What is Grafana, and how does it integrate with Prometheus?

• Grafana is a visualization and dashboarding tool — it doesn't store data; it queries data sources.

• Integration: add Prometheus as a Data Source in Grafana (URL: http://prometheus:9090).

• Grafana uses PromQL to query Prometheus and renders results as graphs, heatmaps, gauges, and tables.

• Pre-built dashboards available from grafana.com/dashboards (e.g., Node Exporter, Kubernetes cluster dashboards).

• Grafana also supports: Loki (logs), Tempo (traces), Datadog, CloudWatch, Elasticsearch.

Q17. How do you configure alerts in Grafana?

- Two approaches: Grafana Alerting (built-in, recommended) and Prometheus Alertmanager.
- Grafana Alerting: define alert rules on panels with PromQL conditions; set evaluation intervals.
- Configure Contact Points: Slack, PagerDuty, email, OpsGenie — where alerts are sent.
- Create Notification Policies to route different alert severities to different channels.
- Example rule: fire alert if avg CPU > 80% for 5 minutes → notify Slack #ops channel.

■ *Always set pending period (e.g., 5m) to avoid alert flapping on transient spikes.*

Q18. What is Grafana Loki, and how is it used for logging?

- Loki is Grafana's log aggregation system — designed to be cost-efficient by indexing only labels, not full log content.
- Architecture: Promtail (agent on each node) ships logs → Loki stores them → Grafana queries via LogQL.
- Labels in Loki mirror Kubernetes labels (namespace, pod, container) for easy correlation.
- LogQL syntax: {namespace='prod', app='api'} |= 'ERROR' — filter logs like grep but with label context.
- Unlike ELK, Loki does not full-text index logs — makes it cheaper but slower for substring searches.

■ *Loki + Prometheus + Tempo = the Grafana observability stack (logs, metrics, traces) — cost-effective alternative to ELK + Jaeger.*

SECTION 3 — Kubernetes: Sidecar, Init Containers & Eviction

Q19. What is a Sidecar container?

- A Sidecar is a secondary container running in the same Pod as the main application container.
- It shares the same network namespace (localhost) and can share volumes with the main container.
- It extends or enhances the main container without modifying its code.
- Common in service mesh (Envoy sidecar in Istio) and observability patterns.

■ *Kubernetes 1.29+ introduced native sidecar support (restartPolicy: Always in initContainers) for lifecycle ordering.*

Q20. What are common use cases of Sidecar containers?

- Log shipping: Filebeat/Fluentd sidecar tails app logs and ships to Elasticsearch/Loki.
- Service mesh proxy: Envoy (Istio) handles mTLS, retries, circuit breaking transparently.
- Secrets injection: Vault Agent sidecar fetches and refreshes secrets into a shared volume.
- Config sync: sidecar pulls config from a remote source and writes to shared volume.
- Metrics exporter: sidecar exposes /metrics for an app that doesn't natively support Prometheus.

Q21. What is an Init Container?

- An Init Container is a specialized container that runs to completion before any main containers start.
- Multiple init containers run sequentially — each must succeed before the next starts.
- Used to perform setup tasks: migrations, dependency checks, config generation.
- If an init container fails, the Pod restarts based on its restartPolicy.

```
initContainers:
- name: wait-for-db
image: busybox
command: ['sh','-c','until nc -z db 5432; do sleep 2; done']
```

Q22. Difference between Init Container and the main container?

- Lifecycle: Init containers run sequentially before app starts; main containers run in parallel after inits finish.
- Purpose: Init = setup/bootstrap; Main = serve traffic.
- Restart: If init fails, pod restarts from init phase. If main fails, only that container restarts (if configured).
- Resources: Init container resource limits are independent — useful for migration jobs needing more memory.
- Health probes: Init containers do not support readiness/liveness probes (they must exit 0 to succeed).

Q23. What is Eviction Policy in Kubernetes?

- Eviction is the process where the kubelet terminates pods to reclaim node resources (CPU, memory, disk).
- Hard eviction threshold: node immediately evicts pods when a threshold is crossed (e.g., memory.available < 100Mi).
- Soft eviction threshold: node waits a grace period before evicting (e.g., memory.available < 200Mi for 2m).
- Priority: pods with no resource requests, BestEffort QoS, and those exceeding limits are evicted first.
- QoS classes: Guaranteed (requests=limits) > Burstable > BestEffort — Guaranteed pods are last to be evicted.

■ *Always set resource requests and limits. Pods without requests are BestEffort and first to be killed.*

Q24. Why do pods get evicted?

- Node memory pressure: system runs out of memory, kubelet kills pods to free it.
- Node disk pressure: ephemeral storage (logs, tmp files) fills up beyond threshold.
- PID pressure: too many processes on the node.
- Preemption: higher-priority Pod needs resources — lower-priority pods get evicted.
- Manual eviction: kubectl drain node or cluster autoscaler downscaling a node.

■ *Check: kubectl describe node | grep -A5 Conditions — look for MemoryPressure/DiskPressure.*

SECTION 4 — Troubleshooting: Pods, Logs & CrashLoopBackOff

Q25. How do you check the logs of a crashed pod?

- Current logs: kubectl logs -n
- Specific container: kubectl logs -c
- Previous (crashed) container: kubectl logs --previous
- Describe for events: kubectl describe pod — shows last state, exit code, reason.

```
# Get last 100 lines from crashed container
kubectl logs --previous --tail=100
```

■ *Exit code 137 = OOMKilled; Exit code 1 = app error; Exit code 139 = segfault.*

Q26. What is the use of kubectl logs --previous?

- --previous fetches logs from the last terminated container instance, not the currently running one.
- Critical when a container crashes and restarts: the new instance starts fresh logs; --previous shows what happened before.
- Only works if the previous container's logs are still in kubelet's log buffer (not purged).
- Combine with -c flag for multi-container pods: kubectl logs --previous -c .

■ *For long-term crash forensics, ship logs to a central store (Loki/ELK) — kubelet log buffers are temporary.*

Q27. How do you troubleshoot a pod crash (CrashLoopBackOff)?

- Step 1: kubectl describe pod — check Events section for OOMKilled, image pull errors, probe failures.
- Step 2: kubectl logs --previous — read the actual application error before crash.
- Step 3: Check resource limits — if memory limit too low, pod gets OOMKilled repeatedly.
- Step 4: Check liveness/readiness probes — misconfigured probes kill healthy containers.
- Step 5: Exec into a debug container: kubectl debug -it --image=busybox -- /bin/sh
- Step 6: Check ConfigMaps/Secrets mount — missing or malformed env vars crash apps on startup.

■ *CrashLoopBackOff = repeated crashes. Backoff increases (10s → 20s → 40s → 5m). The pod IS restarting; the issue is WHY it crashes.*

SECTION 5 — Docker

Q28. What is Docker, and why is it used?

- Docker is a containerization platform that packages an application and its dependencies into a portable container.
- Containers share the host OS kernel — lighter than VMs, start in milliseconds.
- Key benefits: environment consistency (dev = staging = prod), isolation, fast deployment, easy scaling.
- Docker uses Linux namespaces (isolation) and cgroups (resource limits) under the hood.

■ *Docker is the packaging layer. Kubernetes is the orchestration layer. They are complementary, not competing.*

Q29. What is a Docker image?

- A Docker image is a read-only, layered filesystem snapshot containing the app, runtime, and dependencies.
- Images are built from Dockerfiles and stored in registries (Docker Hub, ECR, GCR).
- Each instruction in a Dockerfile creates a new layer — layers are cached for faster rebuilds.
- Images are immutable — a running instance of an image is a container.
- Image naming: registry/repo:tag — e.g., 123.dkr.ecr.ap-south-1.amazonaws.com/apnakart:v1.2.3

Q30. What is a Dockerfile?

- A Dockerfile is a text file containing instructions to build a Docker image layer by layer.
- Key instructions: FROM (base image), RUN (execute command), COPY (add files), CMD/ENTRYPOINT (startup command), EXPOSE (document port).

```
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY target/service.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

- Multi-stage builds: use a builder image to compile, then copy only the artifact to a slim runtime image.

■ *Always use specific image tags (not :latest) and non-root users (USER 1000) in production Dockerfiles.*

Q31. How do you troubleshoot a Docker build failure?

- Read the error output — Docker shows exactly which RUN step failed and the exit code.
- Use --no-cache flag to rule out stale cached layers: docker build --no-cache .
- Run intermediate steps manually: docker run -it /bin/sh and execute commands one by one.
- Check network issues: pip/npm/apt failures are often DNS or proxy problems inside the build context.
- Verify COPY paths: Docker build context must include the files being copied.

■ *docker build --progress=plain gives verbose output — use it to see full command output instead of collapsed layers.*

Q32. What are common Docker issues (auth, daemon, resources)?

• Auth errors: docker login fails — token expired, registry URL mismatch, or ECR token not refreshed (aws ecr get-login-password).
• Daemon not running: 'Cannot connect to Docker daemon' — run sudo systemctl start docker.
• Disk space: 'No space left on device' — run docker system prune -af to remove unused images/containers.
• Resource limits: container OOMKilled — increase --memory flag or reduce app memory footprint.
• Port conflicts: 'Port already in use' — check ss -tulpn or lsof -i : for conflicting processes.

SECTION 6 — Terraform & Infrastructure as Code

Q33. What is Terraform?

• Terraform is an open-source IaC tool by HashiCorp that provisions infrastructure using declarative HCL configuration files.
• It supports multi-cloud: AWS, GCP, Azure, and 1000+ providers via a plugin ecosystem.
• Core workflow: terraform init → terraform plan → terraform apply → terraform destroy.
• State file (terraform.tfstate) tracks real-world resource state — store remotely in S3 + DynamoDB for teams.
■ *Terraform is idempotent: applying the same config twice results in no changes if infra already matches state.*

Q34. What are Terraform modules?

• A module is a container for multiple Terraform resources grouped together to serve a common purpose.
• Root module: your main working directory. Child modules: reusable sub-configurations called by root.
• Modules have inputs (variables), outputs, and local logic — they're the function equivalent in IaC.
• Public modules available on Terraform Registry (registry.terraform.io) — e.g., terraform-aws-modules/vpc/aws.
■ *Modules reduce duplication and enforce consistency — define once, use across dev/staging/prod environments.*

Q35. How does Terraform help in reusability?

• Write once, use many: define a module for an EKS cluster, reuse it for dev/staging/prod with different variable values.
• Variables make modules configurable — callers pass instance_type, region, tags without touching module internals.
• Outputs expose resource attributes to callers — e.g., output the RDS endpoint so the app module can use it.
• Version pinning: modules can be versioned in a Git tag or Terraform Registry — teams consume stable versions.
• Workspaces: isolate state per environment using the same codebase.

Q36. Give an example of reusable Terraform modules (EC2, S3).

• EC2 Module (modules/ec2/main.tf):

```
resource "aws_instance" "this" {
ami = var.ami_id
instance_type = var.instance_type
tags = merge(var.tags, { Name = var.name })
}
```

• S3 Module (modules/s3/main.tf):

```
resource "aws_s3_bucket" "this" {
bucket = var.bucket_name
tags = var.tags
}
resource "aws_s3_bucket_versioning" "this" {
bucket = aws_s3_bucket.this.id
versioning_configuration { status = "Enabled" }
}
```

• Calling in root (main.tf):

```
module "web_server" {
source = "./modules/ec2"
ami_id = "ami-0abcdef123"
instance_type = "t3.medium"
name = "web-prod"
}
module "assets_bucket" {
source = "./modules/s3"
bucket_name = "apnakart-assets-prod"
}
```

SECTION 7 — Kubernetes RBAC & Access Control

Q37. What is RBAC in Kubernetes?

• RBAC (Role-Based Access Control) controls what actions users and services can perform on Kubernetes resources.

• Core objects: Role/ClusterRole (define permissions), RoleBinding/ClusterRoleBinding (bind to subjects).

• Role: namespace-scoped. ClusterRole: cluster-wide (nodes, PVs, non-namespaced resources).

• Permissions are additive — no deny rules. If no rule grants access, access is denied.

■ *Principle of least privilege: give only the permissions needed, scoped as narrowly as possible.*

Q38. What is a Service Account?

• A ServiceAccount is an identity for processes running inside pods — not for human users.

• Every pod gets a default ServiceAccount if none is specified — token is auto-mounted at /var/run/secrets/kubernetes.io/serviceaccount/token.

• Use RBAC RoleBindings to grant specific permissions to a ServiceAccount.

• IRSA (IAM Roles for Service Accounts) in EKS: bind a Kubernetes SA to an AWS IAM Role for pod-level AWS permissions without node-level credentials.

■ *Disable automountServiceAccountToken for pods that don't need API access — reduces attack surface.*

Q39. How do you control access in Kubernetes?

• Authentication: who are you? — certificate, OIDC token, static bearer token.

• Authorization: what can you do? — RBAC, ABAC, Webhook.

• Admission Control: should this resource be created? — OPA/Gatekeeper, Kyverno policies.

• NetworkPolicy: restrict pod-to-pod communication at L3/L4.

• PodSecurity Standards: restrict privileged containers, host namespaces, capability escalation.

■ *Production checklist: RBAC + NetworkPolicy + PodSecurity + IRSA/Workload Identity — all four layers.*

SECTION 8 — Logging, Deployment Prep & Updates

Q40. What is log masking/log filtering?

- Log masking: replacing sensitive data (PII, tokens, passwords) in logs with a placeholder before they are stored.
- Example: mask credit card numbers, email addresses, or API keys in application logs.
- Implementation: pattern-based regex in the logging framework (Logback, Log4j2 PatternLayout) or at the shipper level (Fluentd/Logstash filters).
- Log filtering: dropping or routing specific log entries — e.g., filter out health-check /healthz logs to reduce noise.

■ *GDPR/PCI compliance requires log masking — never log raw tokens, passwords, or credit card data. Audit this regularly.*

Q41. What are the steps to prepare an application before deployment?

- 1. Code review and static analysis (SonarQube) — catch bugs and code smells early.
- 2. Unit and integration tests — fail fast before building the image.
- 3. Docker image build + vulnerability scan (Trivy) — reject if critical CVEs found.
- 4. Push image to registry with a versioned tag (never :latest in prod).
- 5. Update Kubernetes manifests or Helm values with the new image tag.
- 6. Run terraform plan if infra changes are included.
- 7. Deploy to lower environment first (dev/staging) and run smoke tests.
- 8. Promote to production with approval gate.

■ *In ApnaKart: Jenkins pipeline implements all these stages; SonarQube quality gate and Trivy zero-critical gate must pass before ECR push.*

Q42. How do you handle dependencies before container startup?

- Init containers: run a wait-for-db or wait-for-kafka init container that loops until dependency is ready.

```
command: ['sh','-c','until nc -z kafka 9092; do echo waiting; sleep 3; done']
```

- readinessProbe: the main container doesn't receive traffic until its readiness probe passes.
- Application-level retry: build retry logic with backoff into the app startup code (resilience4j, spring-retry).
- Helm hooks: pre-install/pre-upgrade hooks run Jobs (e.g., DB migrations) before the main deployment rolls out.

■ *Don't rely solely on init containers for DB migrations in production — use dedicated migration jobs or Helm hooks for better control and rollback.*

Q43. What is a Rolling Update in Kubernetes?

- A Rolling Update replaces old pods with new pods incrementally — zero downtime deployments.
- Controlled by maxSurge (extra pods allowed during update) and maxUnavailable (pods that can be down).
- Kubernetes waits for new pods to pass readiness before terminating old ones.

```
strategy:
type: RollingUpdate
rollingUpdate:
maxSurge: 1
maxUnavailable: 0
```

- kubectl rollout status deployment/ — monitors progress.
- kubectl rollout undo deployment/ — instant rollback to the previous ReplicaSet.

■ *maxUnavailable: 0 + maxSurge: 1 = safest rolling update (always has full capacity). Costs extra resources during update.*

Q44. How do you perform updates using Helm charts?

- Update image tag in values.yaml or pass it via --set flag.
- Dry run: helm upgrade --dry-run --debug — validate changes without applying.
- Apply: helm upgrade -f values-prod.yaml --set image.tag=v2.3.1
- Atomic flag: --atomic rolls back automatically if the upgrade fails (waits for pods to be ready).
- Rollback: helm rollback — instant rollback to a previous Helm revision.
- History: helm history — lists all revisions with timestamps and status.

■ *--atomic is highly recommended in CI/CD — it prevents half-deployed broken states by auto-rolling back on failure.*
